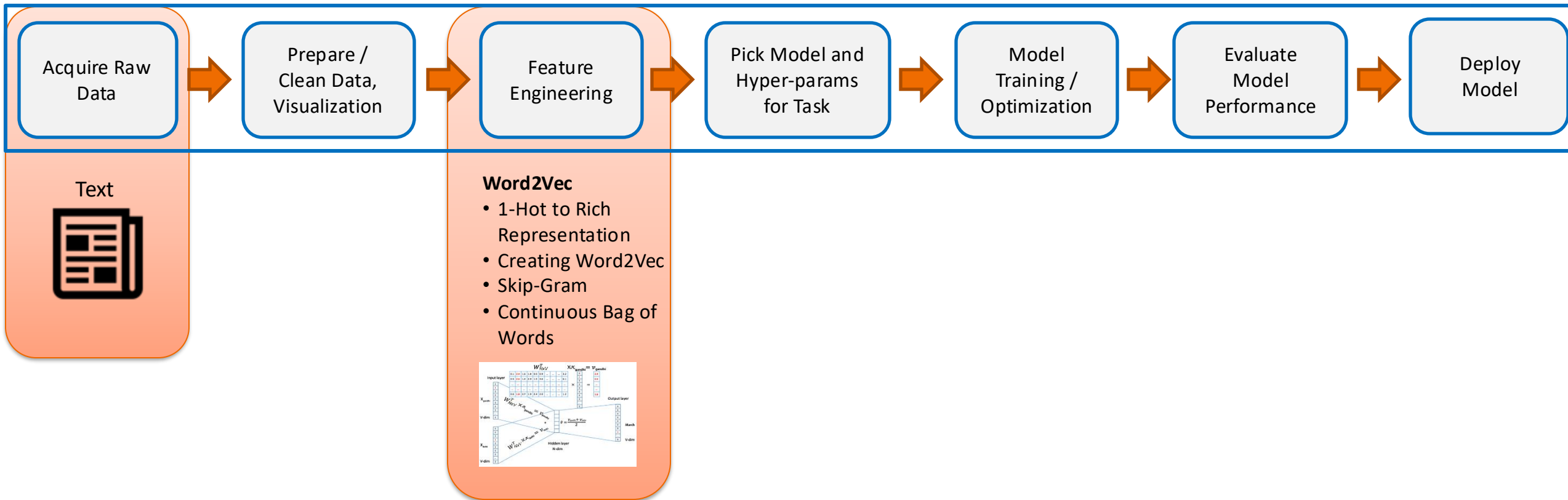


# Focus for this lecture



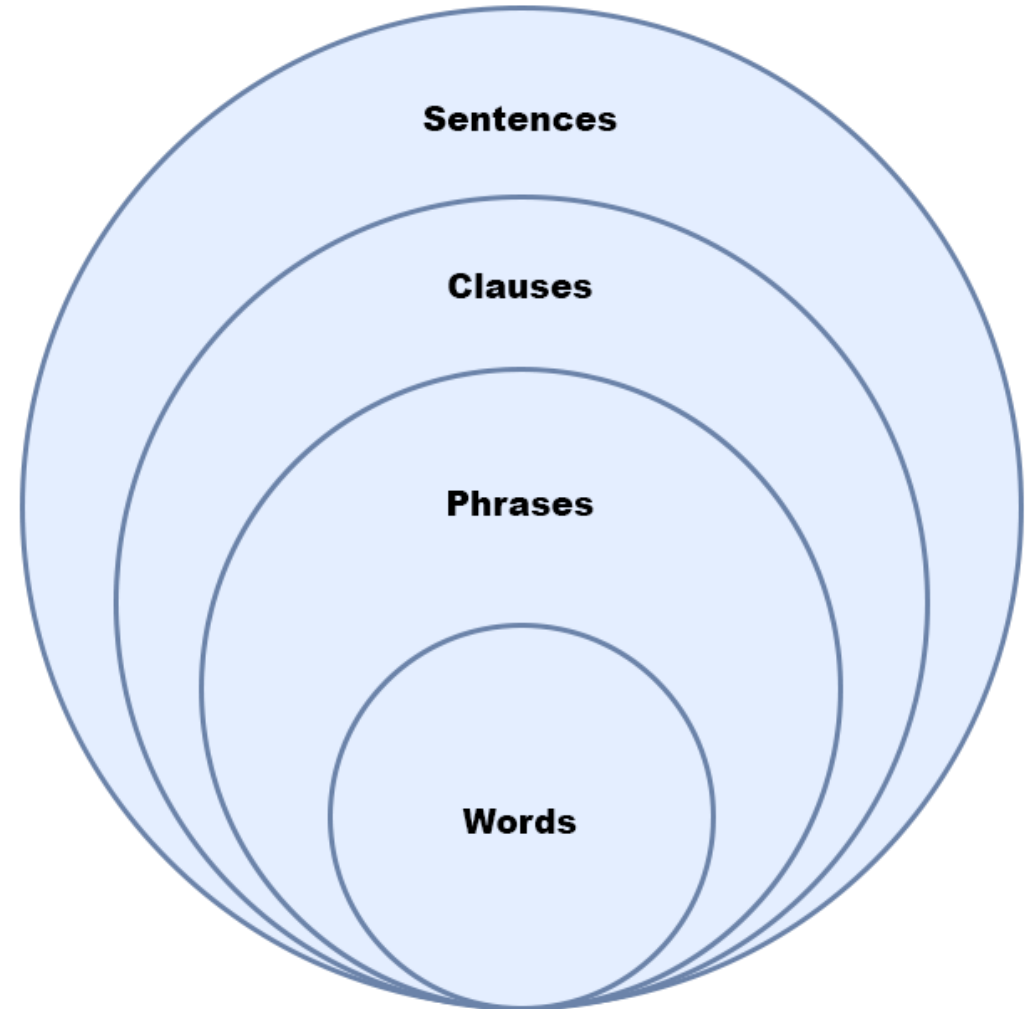
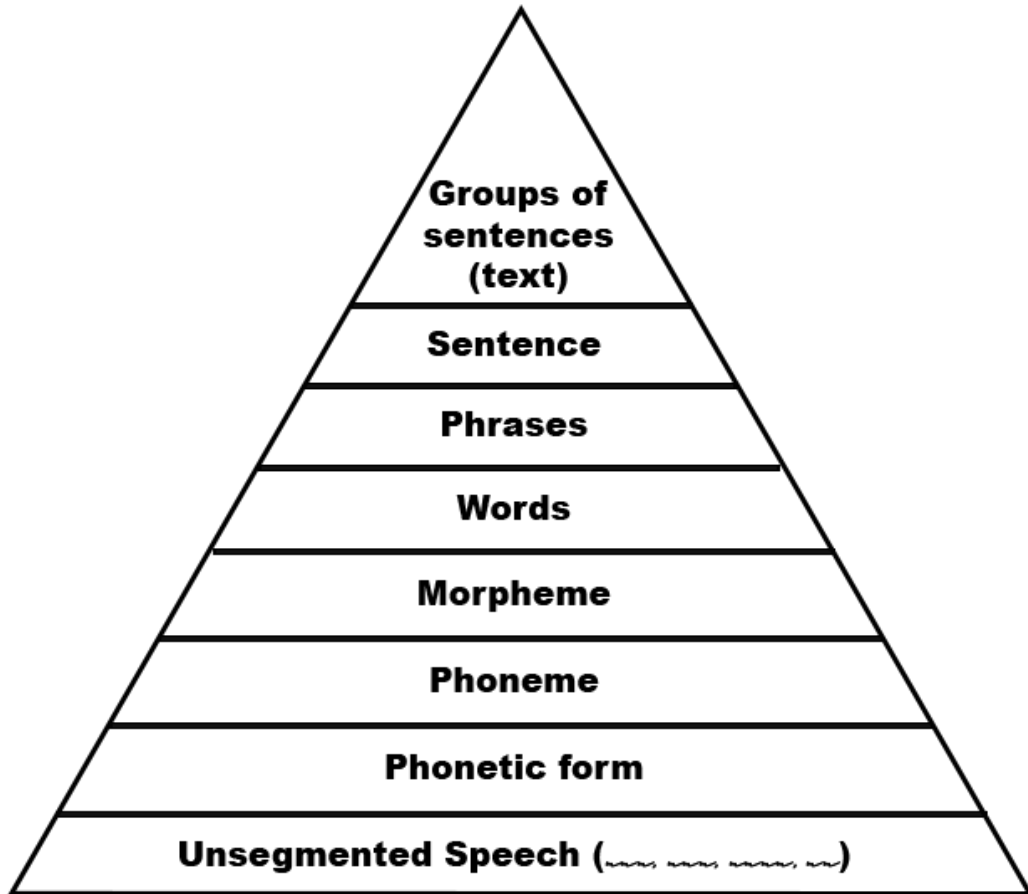
# What Do We Mean by Representing Text?

- Representations for “words”?
- Representations for “phrases”?
- Representations for “sentences”?
- Representations for “documents”?

# Previous Lecture

- Document was represented in terms of word ‘statistics’
  - as given by TF-IDF for each dictionary term
- Observation: Words were not part of document representation explicitly
- **Can we have document representation in terms of word representations? Any advantages?**

# Hierarchical nature of documents



# Word Representation

- Aim: Unique numerical representation for each unique word in the dictionary.
- Idea: Consecutive integers
  - Apple = 1
  - Azaleas = 2
  - ...
  - Zebra = 1,09,200
- Seems ok ?

# Word Representation

- Aim: Unique and **sensible** numerical representation for each unique word in the dictionary.
- Sensible → All words are 'treated the same'

# 1-Hot Representation of a Word

- **D1** : Andrew is a tall boy.
- **D2** : Ram is a good boy. Ratna is also good.

|            |        |                                 |                         |
|------------|--------|---------------------------------|-------------------------|
| Dictionary | A      | $[1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0]^T$ | } 1-Hot representations |
|            | Also   | $[0\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0]^T$ |                         |
|            | Andrew | $[0\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 0]^T$ |                         |
|            | Boy    | $[0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 0]^T$ |                         |
|            | Good   | $[0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0]^T$ |                         |
|            | Tall   | $[0\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 0]^T$ |                         |
|            | Is     | $[0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0]^T$ |                         |
|            | Ram    | $[0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0]^T$ |                         |
|            | Ratna  | $[0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1]^T$ |                         |

# Histogram vs. 1-Hot Representation

- Histogram is the sum of one-hot representation of all the words
- D1:** Andrew is a tall boy

|        | 1-Hot Representation |   |   |   |   |   |   |   |   |   | Histogram (Bag of Words) |
|--------|----------------------|---|---|---|---|---|---|---|---|---|--------------------------|
| A      | 0                    |   | 0 | 1 | 0 | 0 | 0 | 0 | 1 |   | 1                        |
| Also   | 0                    |   | 0 | 0 | 0 | 0 | 0 | 0 | 0 |   | 0                        |
| Andrew | 1                    |   | 0 | 0 | 0 | 0 | 0 | 0 | 1 |   | 1                        |
| Boy    | 0                    |   | 0 | 0 | 0 | 0 | 1 | 0 | 1 |   | 1                        |
| Good   | 0                    | + | 0 | + | 0 | + | 0 | + | 0 | = | 0                        |
| Tall   | 0                    |   | 0 | 0 | 0 | 1 | 0 | 0 | 1 |   | 1                        |
| Is     | 0                    |   | 1 | 0 | 0 | 0 | 0 | 0 | 1 |   | 1                        |
| Ram    | 0                    |   | 0 | 0 | 0 | 0 | 0 | 0 | 0 |   | 0                        |
| Ratna  | 0                    |   | 0 | 0 | 0 | 0 | 0 | 0 | 0 |   | 0                        |



# Document Histogram

- Histogram for both the documents:

|            |        |   |   |
|------------|--------|---|---|
| Dictionary | A      | 1 | 1 |
|            | Also   | 0 | 1 |
|            | Andrew | 1 | 0 |
|            | Boy    | 1 | 1 |
|            | Good   | 0 | 2 |
|            | Tall   | 1 | 0 |
|            | Is     | 1 | 2 |
|            | Ram    | 0 | 1 |
|            | Ratna  | 0 | 1 |

- **D1** : Andrew is a tall boy.
- **D2** : Ram is a good boy.  
Ratna is also good.

# Representing Words

- Words are atomic entities
- One-hot representation does not capture the meaning
  - Words with similar or different meanings are equally apart
  - Cosine distance is 0 between any two words
  - This is unlike apples and oranges
- Can words be vectors with meaningful distances?
  - How do we capture meaning?
  - How do we learn this from unlabeled data?

# Word2Vec

- $\text{Man} - \text{Woman} + \text{Queen} = \text{King!!}$

# Intuition

- Marco saw a furry little wampimuk hiding in the tree

- **What is Marco?**
- **What is wampimuk?**



A person



Animal

**If I know the meaning of the surrounding words, I  
know the meaning of this word**

- Lets make it work

# Word2Vec: Comments

- Meaning of a word is captured by the Co-occurring words.
- Example
  - I traveled by **Bus**. I traveled by **Car**
  - Bus and Car should have similar representations but not identical.
- **Why?**
- Example
  - This is our family **Car**. This is a public **Bus**.

# What Do We Want? Ans:

- Learn a representation such that:
- Representation of the middle word is the sum of representations of the left and right words.

$$x_3 = f(W, x_1, x_2, x_4, x_5)$$

$$x_1 = R(a_1)$$

$$x_2 = R(a_2)$$

$$x_3 = R(a_3)$$

$$x_4 = R(a_4)$$

$$x_5 = R(a_5)$$

*For all sentences (assume sentences are pentalets)*

*We also need learnable parameters/weights,  $W$  (matrices)*

*$a_i$  is the  $i^{\text{th}}$  word;  $R(a_i)$  is a representation of the word*

# Problem: Classify Text

- T1: Gandhi went on a strike.
- T2: Bapu went on a march.
- T3: Many joined the strike in March.
- T4: The Netherlands did not qualify.
- T5: The Dutch were upset.



# Consider Two Sentences

- Gandhi went on a **march** at Delhi.
- Gandhi went on a **strike** at Delhi.
  - Their 1-hot representations are different as the machine does not understand that “**march**” has something to do with “**strike**”.

# Consider Two Sentences

- Gandhi went on a **march** at Delhi.
- Gandhi went on a **strike** at Delhi.
- We represent these as triplets (remove stop words):
  - $\langle \text{Gandhi, March, Delhi} \rangle$  and  $\langle \text{Gandhi, Strike, Delhi} \rangle$
  - In general:  $\langle a_1, a_2, a_3 \rangle$
  - (We will relax the three-word sentences later.)

# What Do We Want?

- We want the representation of **Strike** and **March** to be the same (or at least very similar). How?
  - We will learn such a “nice” “magic” representation. (details follow)
  - **Intuition:** We know that meaning of a word is defined by the co-occurring words.

# Consider The Five Sentences:

- Gandhi resisted evil with ahimsa for independence
- Bapu fought evil with pacifism for freedom
- Gandhi combated iniquity with nonviolence for liberty
- Mahatma struggled against vice with ahimsa for freedom
- Gandhi says confrontation against evil needs ahimsa for swarajya

# If We Remove Stop Words

- <Gandhi, resisted, evil, ahimsa, independence>
- <Bapu, fought, evil, pacifism, freedom>
- <Gandhi, combated, iniquity, nonviolence, liberty>
- <Mahatma, struggled, vice, ahimsa, freedom>
- <Gandhi, confrontation, evil, ahimsa, swarajya>
- In general:  $\langle a_1, a_2, a_3, a_4, a_5 \rangle$

# What Do We Want?

- We want the representation to be the same (or at least very similar) for:
  - Gandhi, Bapu, and Mahatma
  - Resisted, fought, combated, struggled, confrontation
  - Evil, iniquity, vice
  - Nonviolence, pacifism, ahimsa
  - Independence, freedom, liberty, swarajya

# What Do We Want? Ans:

- Learn a representation such that:
- **Representation of the middle word is the sum of representations of the left and right words.**

$$R(a_3) = F(R(a_1), R(a_2), R(a_4), R(a_5))$$

*For all sentences (assume sentences are pentalets)*

# What Do We Want? Ans:

- Learn a representation such that:
- **Representation of the middle word is the sum of representations of the left and right words.**

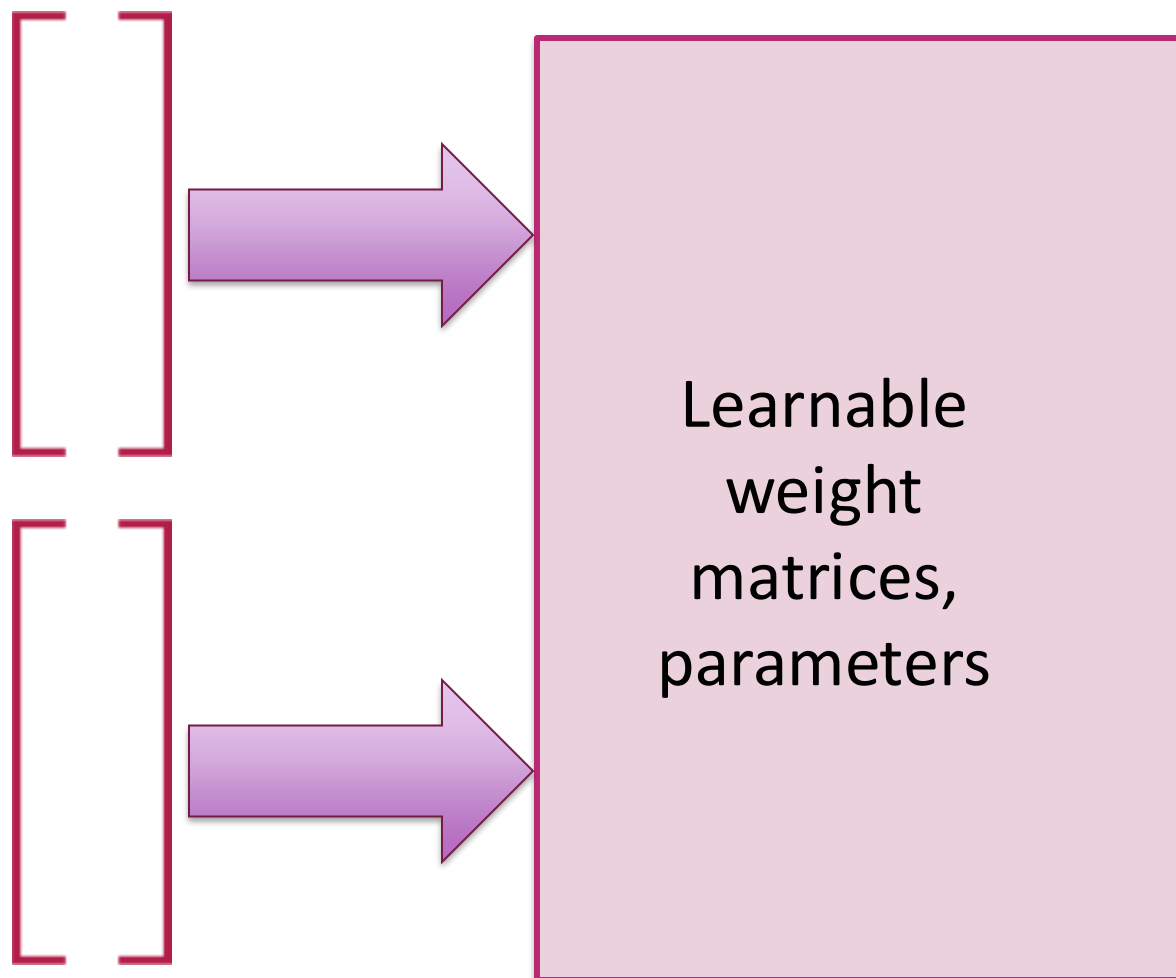
$$R(a_3) = F(W, R(a_1), R(a_2), R(a_4), R(a_5))$$

*For all sentences (assume sentences are pentalets)*

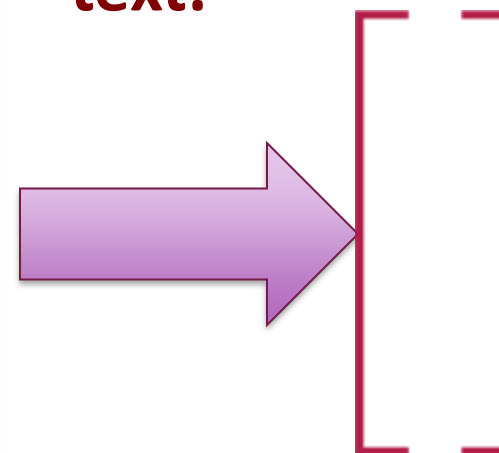
*We also need learnable parameters/weights,  $W$  (matrices)*



# Summary



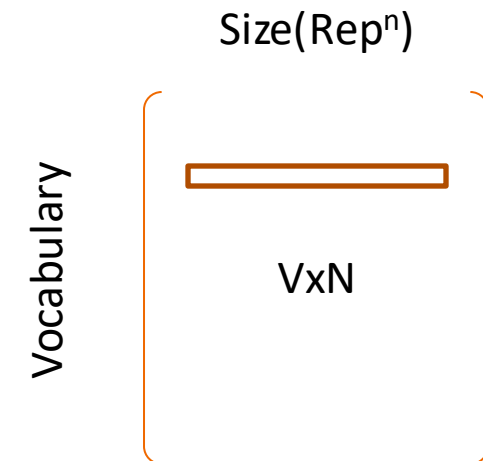
**Problem: Learn the parameters, such that this is (approximately) “TRUE” for all pentalets in the text.**



**Solution: We train with a large text corpus and find such a  $W$**

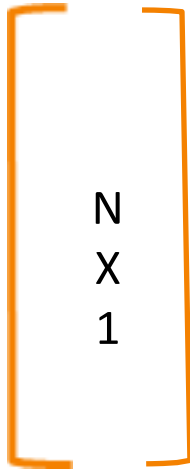
# Detail

- Weight matrices are of size  $V \times N$
- $V$  is the vocabulary size;  $N$  is the size of the new representation
- For the  $i^{\text{th}}$  word in the vocabulary, representation is the  $i^{\text{th}}$  row of the matrix.



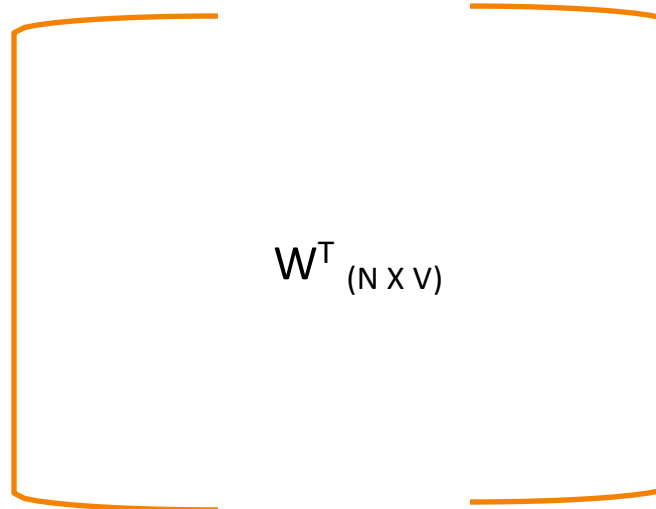
# Creating the Word2Vec

Dense;  
Semantic



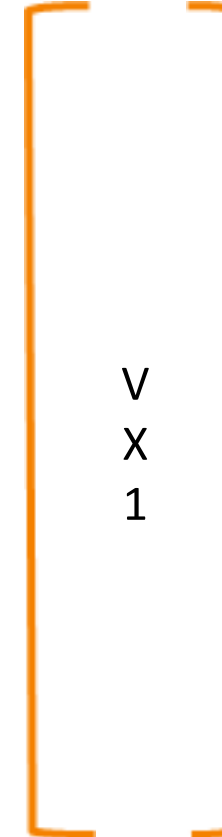
=

Learned Matrix



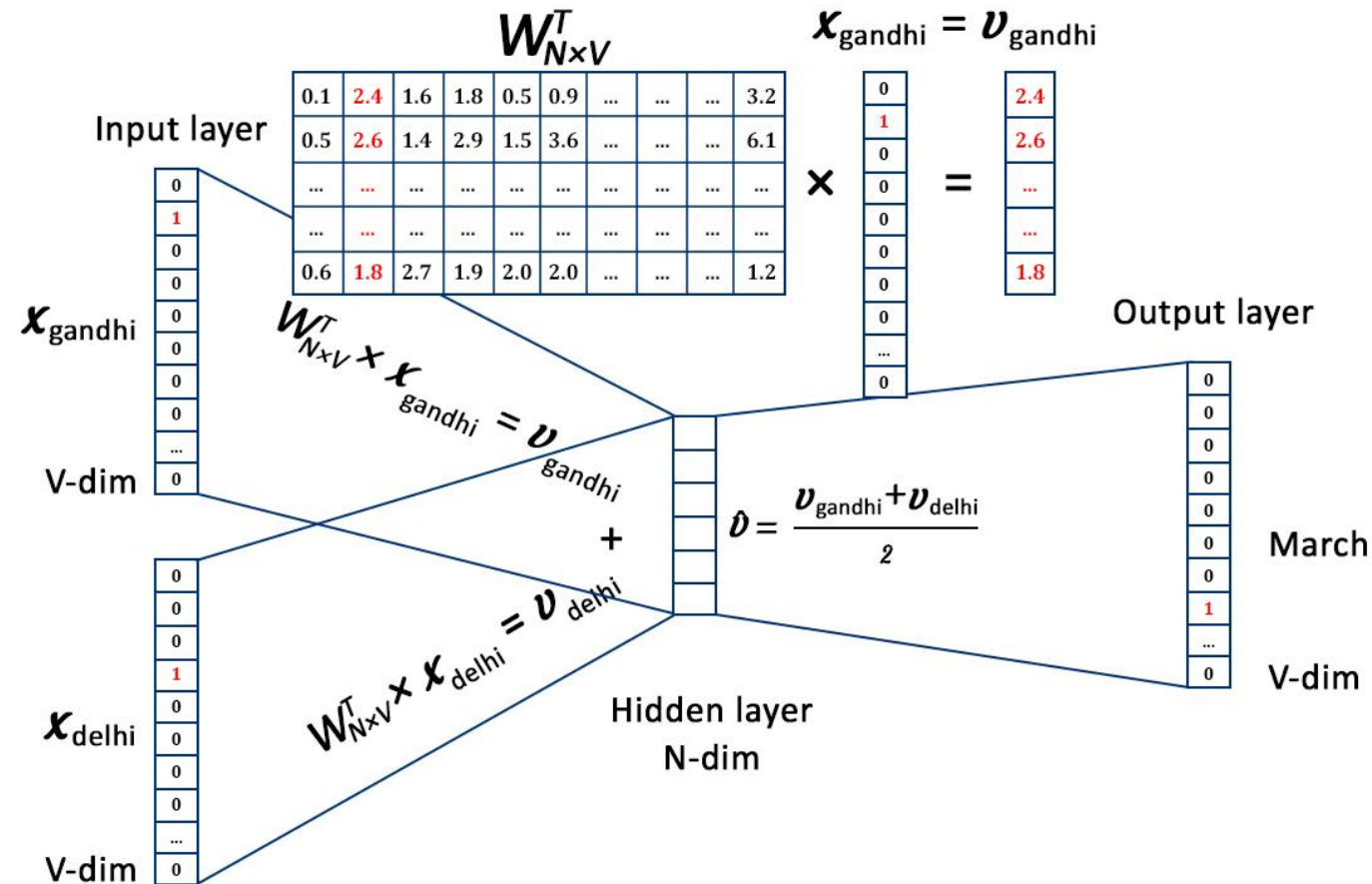
Use  
Precomputed  
Matrix

One hot;  
Sparse



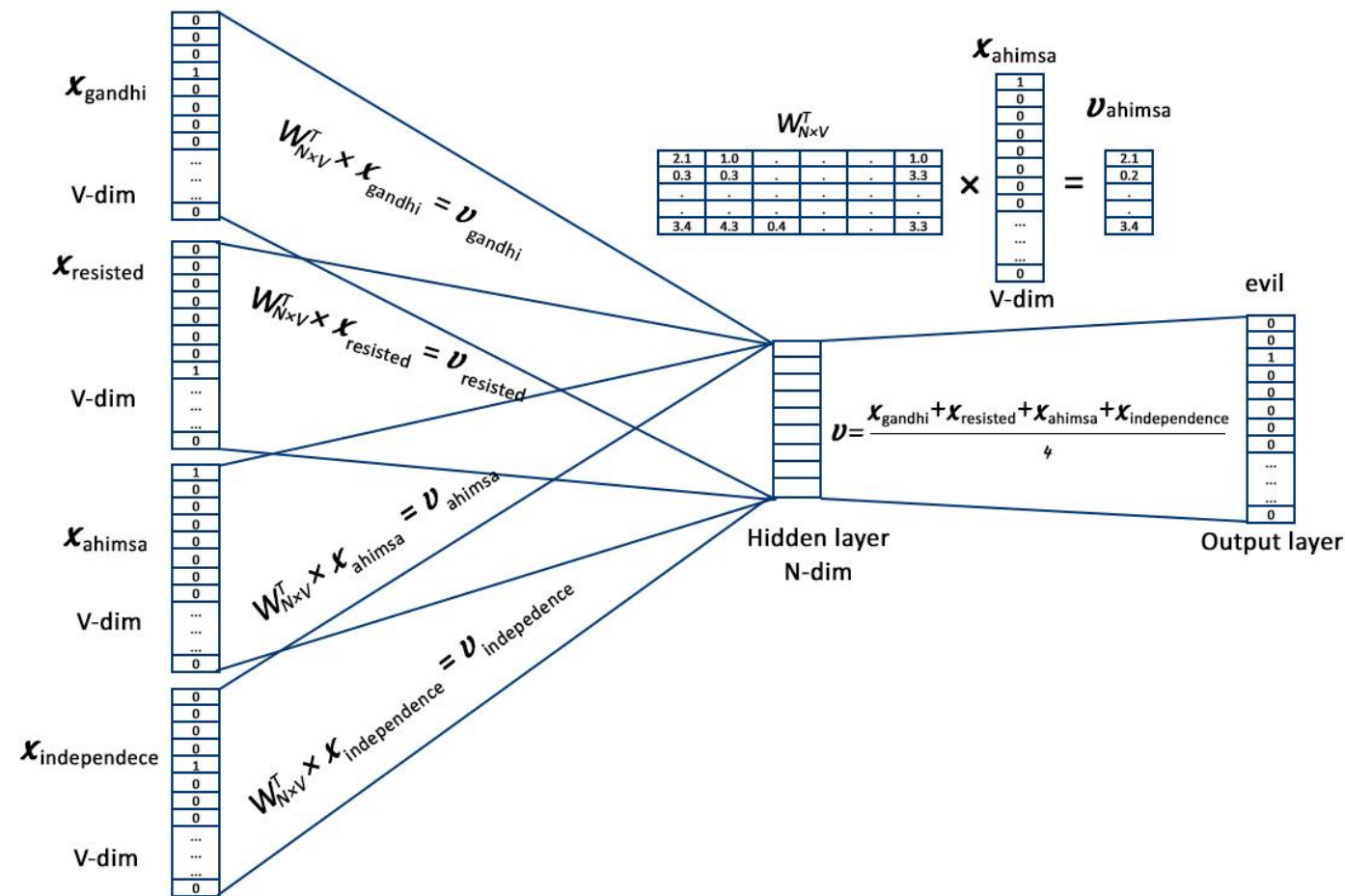
# More Closely (Eg. <gandhi, march, delhi>)

Average to be replaced with f



# Detail (eg. <gandhi, resisted, ahimsa, independence>)

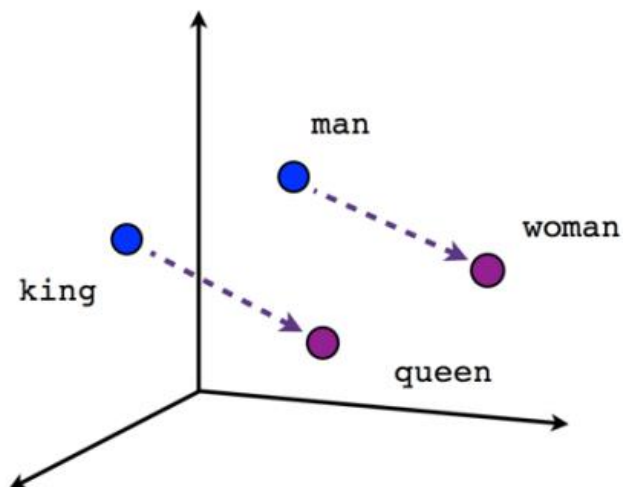
Average to be replaced with f



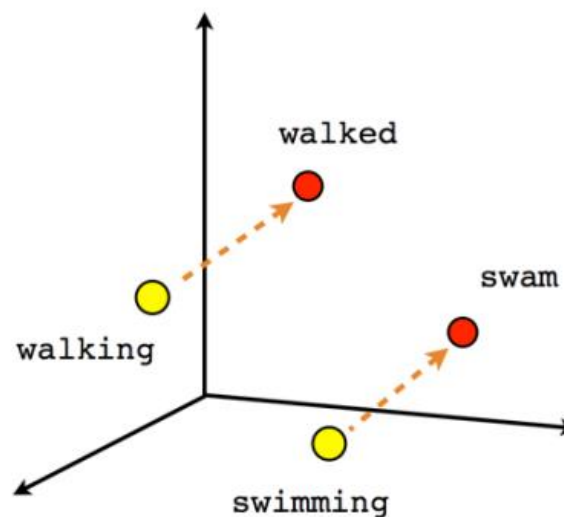
# 1-Hot to Rich Representations

| Word  | One-Hot Encoding   | Word2Vec Embedding                          |
|-------|--------------------|---------------------------------------------|
| Bull  | [1, 0, 0, 0, 0, 0] | [0.95, 1.2, -1.9, -0.008, 0.005, 0.00]      |
| Cow   | [0, 1, 0, 0, 0, 0] | [0.943, 0.922, 1.27, 0.302, -1.07, -4.86]   |
| Eagle | [0, 0, 1, 0, 0, 0] | [-2.88, 6.74, 4.15, -.001, -0.0002, 1]      |
| Goat  | [0, 0, 0, 1, 0, 0] | [0.71, 0.25, 1.004, 0.377, 1.34, 0.000]     |
| Lion  | [0, 0, 0, 0, 1, 0] | [0.299, -1.08, 0.100, -1.85, -0.0067, 0.00] |
| Tiger | [0, 0, 0, 0, 0, 1] | [-0.003, -1.97, -0.49, 1.2, -0.257, 0.000]  |

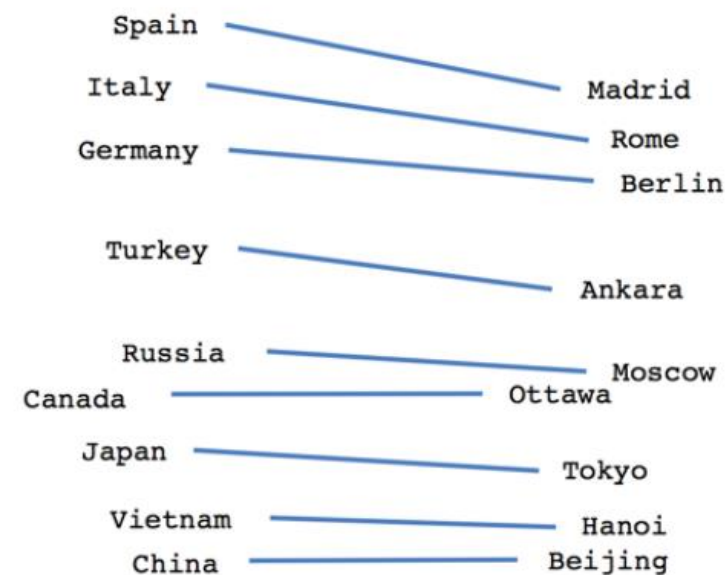
# Examples



Male-Female

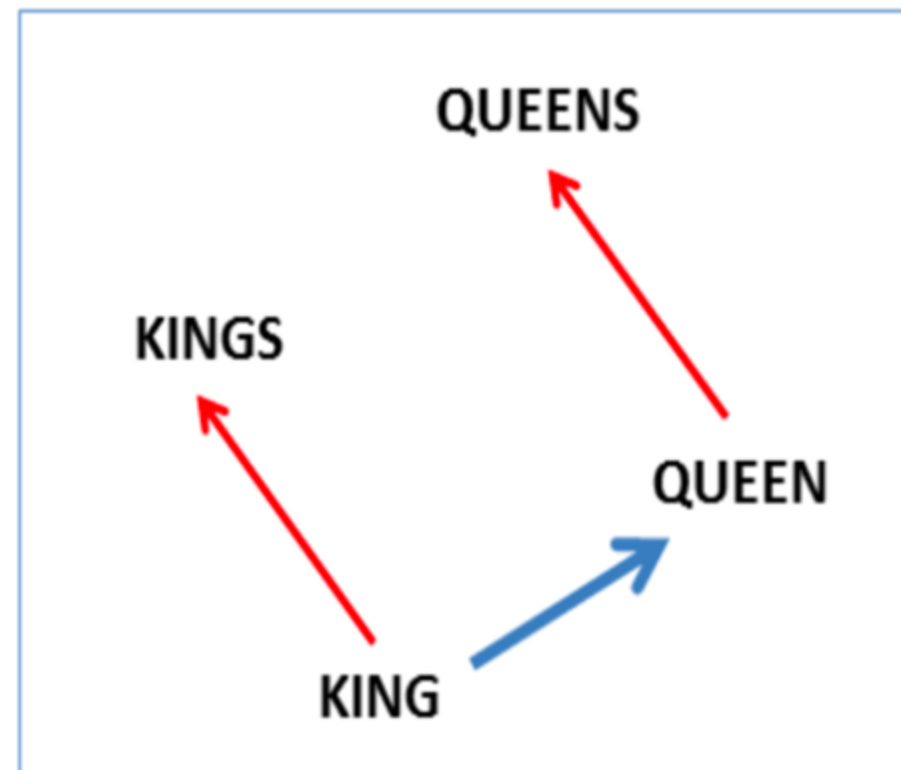
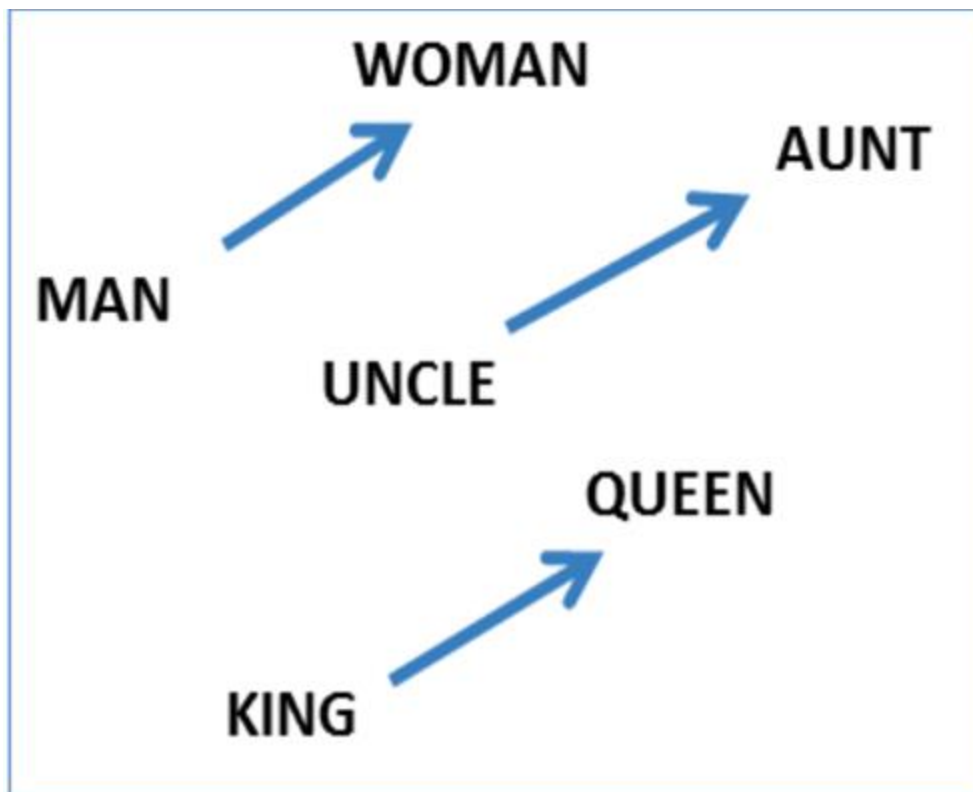


Verb tense



Country-Capital

# Word Similarity = Vector Similarity



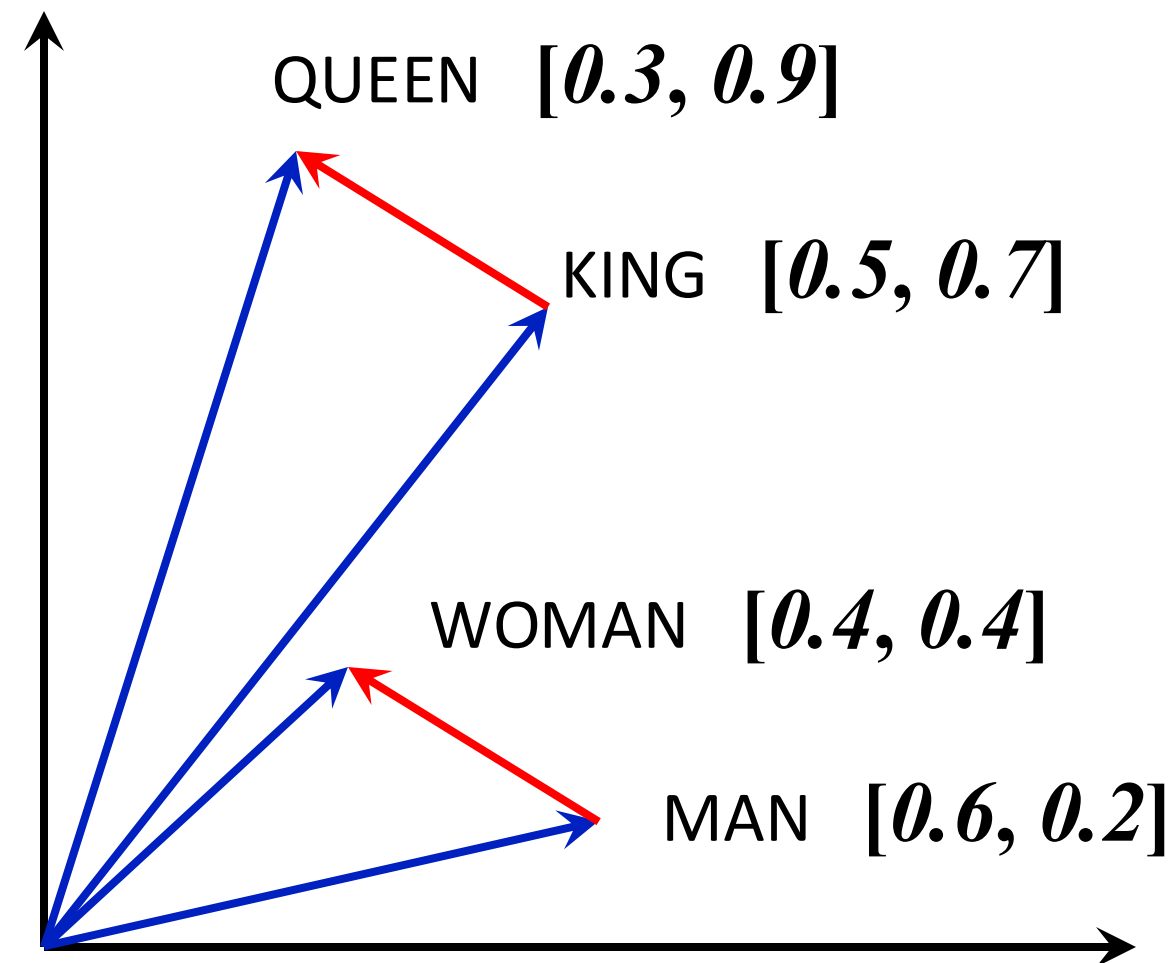
(Mikolov et al., NAACL HLT, 2013)



# Visualising “Word-Arithmetic”

$$\begin{aligned} \text{Vec}(\text{Queen}) \\ = \text{Vec}(\text{King}) - \text{Vec}(\text{Man}) + \text{Vec}(\text{Woman}) \end{aligned}$$

- These fancy arithmetic were imagined (and shown) earlier also; but became popular with Word2Vec.



# More Examples

| FRANCE      | JESUS   | XBOX        | REDDISH   | SCRATCHED | MEGABITS   |
|-------------|---------|-------------|-----------|-----------|------------|
| AUSTRIA     | GOD     | AMIGA       | GREENISH  | NAILED    | OCTETS     |
| BELGIUM     | SATI    | PLAYSTATION | BLUISH    | SMASHED   | MB/S       |
| GERMANY     | CHRIST  | MSX         | PINKISH   | PUNCHED   | BIT/S      |
| ITALY       | SATAN   | IPOD        | PURPLISH  | POPPED    | BAUD       |
| GREECE      | KALI    | SEGA        | BROWNISH  | CRIMPED   | CARATS     |
| SWEDEN      | INDRA   | PSNUMBER    | GREYISH   | SCRAPED   | KBIT/S     |
| NORWAY      | VISHNU  | HD          | GRAYISH   | SCREWED   | MEGAHERTZ  |
| EUROPE      | ANANDA  | DREAMCAST   | WHITISH   | SECTIONED | MEGAPIXELS |
| HUNGARY     | PARVATI | GEFORCE     | SILVERY   | SLASHED   | GBIT/S     |
| SWITZERLAND | GRACE   | CAPCOM      | YELLOWISH | RIPPED    | AMPERES    |

What words have embeddings closest to a given word? From Collobert et al. (2011) (<https://arxiv.org/pdf/1103.0398v1.pdf>)

# More Examples

| Relationship         | Example 1           | Example 2         | Example 3            |
|----------------------|---------------------|-------------------|----------------------|
| France - Paris       | Italy: Rome         | Japan: Tokyo      | Florida: Tallahassee |
| big - bigger         | small: larger       | cold: colder      | quick: quicker       |
| Miami - Florida      | Baltimore: Maryland | Dallas: Texas     | Kona: Hawaii         |
| Einstein - scientist | Messi: midfielder   | Mozart: violinist | Picasso: painter     |
| Sarkozy - France     | Berlusconi: Italy   | Merkel: Germany   | Koizumi: Japan       |
| copper - Cu          | zinc: Zn            | gold: Au          | uranium: plutonium   |
| Berlusconi - Silvio  | Sarkozy: Nicolas    | Putin: Medvedev   | Obama: Barack        |
| Microsoft - Windows  | Google: Android     | IBM: Linux        | Apple: iPhone        |
| Microsoft - Ballmer  | Google: Yahoo       | IBM: McNealy      | Apple: Jobs          |
| Japan - sushi        | Germany: bratwurst  | France: tapas     | USA: pizza           |

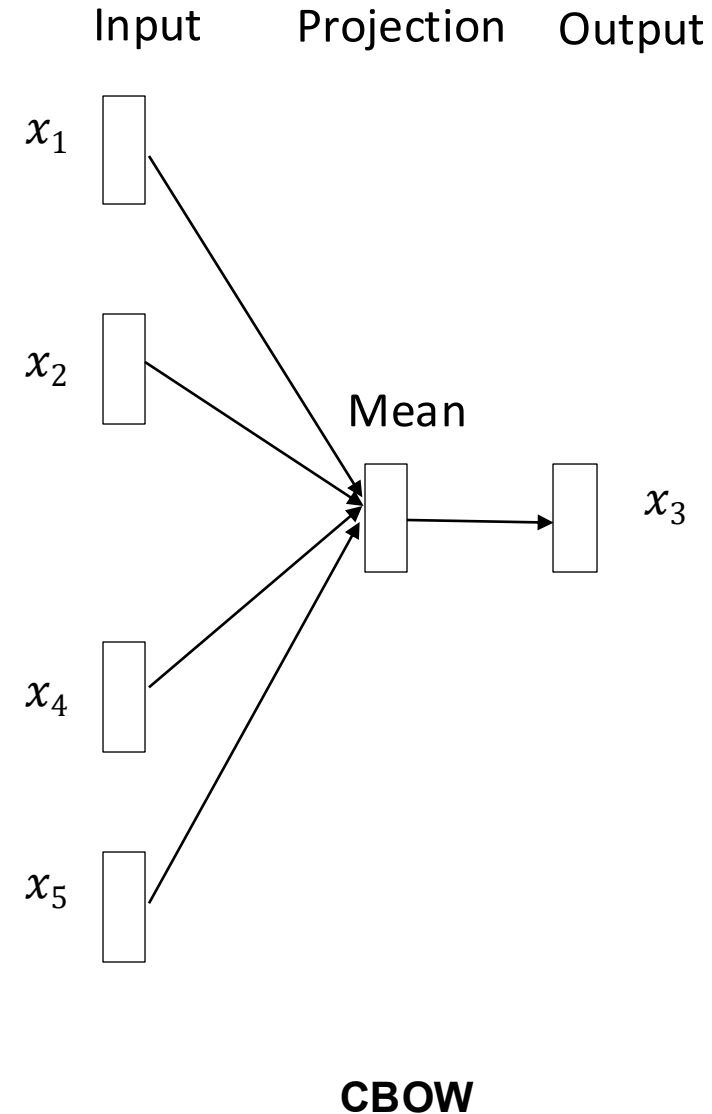
Relationship pairs in a word embedding. From Mikolov et al. (2013b)  
(<https://arxiv.org/pdf/1301.3781.pdf>)

**talent sprint**  
Part of **Accenture**

<https://projector.tensorflow.org/>

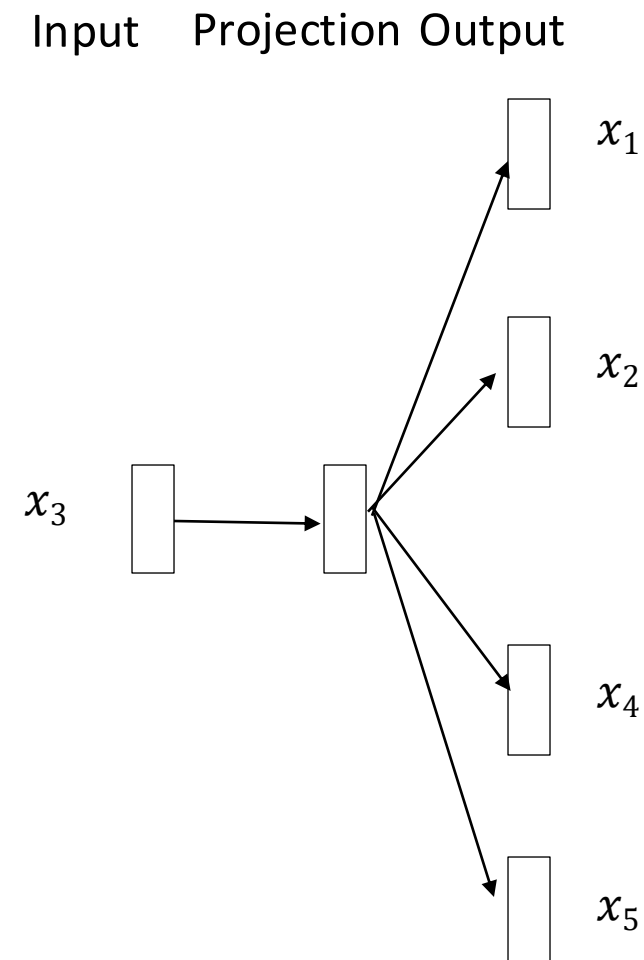
# Variants

- Continuous Bag of Words (CBOW):
  - use a window of words to predict the missing word



# Variants

- Skip-gram (SG):
  - use a word to predict the surrounding ones in the specified window.



# Representing documents (email/web page)

- Word2Vec behaves like a vector
- Vector operations can be done on words
- → Representation for the document can be defined as sum/average of word2Vecs of all words in the document

# Comment: Categorical Variables

- We are worried that we also have “Categorical Variables”
  - Text is a classical example
  - A 1-hot representation is a possibility
- However, we can now have Dense embedding/representations
  - Motivated by Word2Vec



# Some Questions

- How do I use/ extend in my domain?
  - Every problem has its own characteristics. Let us not assume that we can copy solutions.
  - There are numerous examples of generic notions getting employed or extended to your notion. (please see next slide)

# Representing Complex Entities

- Products:
  - Product has attributes
- Represent items as vectors in the same space and use their distances to compute recommendations

# Representing Complex Entities

- Many recent works:
  - Prod2Vec: Sequence of Products
  - MetaProd2Vec: (categories, brand, tags, etc ..)
- Medical Records
  - Visits, Numerical attributes, Personal attributes, Images, Reports

# Representing Complex Entities

- Data:
  - **Visits:** The sequence in which a patient goes to a hospital
  - Medical terms (diagnosis, disease, medication, procedure)

# “X”2Vec

- Motivated by Word2Vec
  - Node2vec
  - Doc2vec
  - See: <https://github.com/MaxwellRebo/awesome-2vec>
- Learn from domain examples
- An embedding that can be manipulated

# Problems with 1-Hot encoding

- The number of dimensions increases as we add words to the vocabulary.
  - For a vocabulary of 50,000 words, each word is represented with 49,999 zeros, and a single “one” value in the correct location. So, memory use is prohibitively large.
- The embedding matrix is very sparse, mainly made up of zeros
- Not useful in finding relationships between words

# Example

- King  $[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]^T$
- Queen  $[0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]^T$
- Water  $[0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0]^T$
- Man  $[0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]^T$
- Woman  $[0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]^T$
- **Euclidean Distance between any two words =  $\sqrt{2}$**
- **Cosine Similarity between any two words = 0**

# But a dense representation will be useful!

- King [0.50451 , 0.68607 , -0.59517 ..... 0.78321 , -0.91241 , -1.6106 ]
- Queen [3.37854 , 1.8233 , -1.2648 ..... -2.2839 , 0.0092753, -0.60284]
- Water [0.53507 , 0.5761 , -0.054351 ..... -0.78654 , 0.61563 , -0.95478]
- Man [-0.094386, 0.43007 , -0.17224 ..... -0.97925 , 0.53135 , -0.11725]
- Woman [-0.18153, 0.64827, -0.58210 ..... -1.3138, 0.48764, -1.0467]

Euclidean distance of words

|       | king     | queen    | water    | man      | woman    |
|-------|----------|----------|----------|----------|----------|
| king  | 0.000000 | 3.477756 | 7.025455 | 5.143857 | 5.942768 |
| queen | 3.477756 | 0.000000 | 6.690359 | 5.023818 | 4.825779 |
| water | 7.025455 | 6.690359 | 0.000000 | 6.235551 | 6.782816 |
| man   | 5.143857 | 5.023818 | 6.235551 | 0.000000 | 2.602624 |
| woman | 5.942768 | 4.825779 | 6.782816 | 2.602624 | 0.000000 |

Cosine similarity of words

|       | king     | queen    | water    | man      | woman    |
|-------|----------|----------|----------|----------|----------|
| king  | 1.000000 | 0.783904 | 0.235159 | 0.530938 | 0.411338 |
| queen | 0.783904 | 1.000000 | 0.286606 | 0.536670 | 0.600311 |
| water | 0.235159 | 0.286606 | 1.000000 | 0.386427 | 0.310141 |
| man   | 0.530938 | 0.536670 | 0.386427 | 1.000000 | 0.886034 |
| woman | 0.411338 | 0.600311 | 0.310141 | 0.886034 | 1.000000 |



# Word vectors - Glove

# Global Vectors (GloVe)

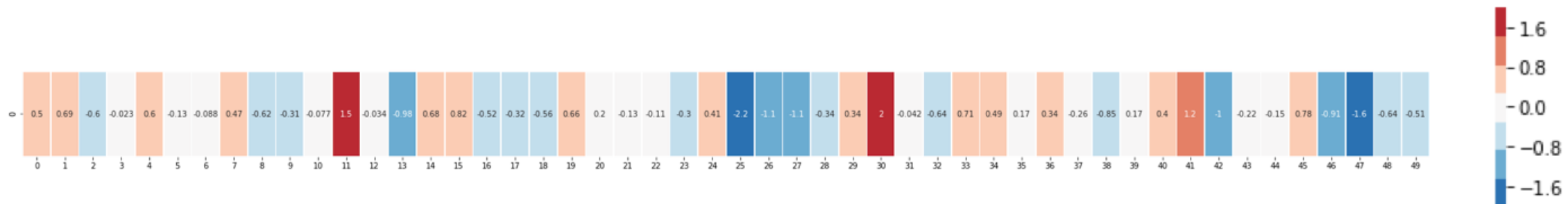
- Stanford NLP's GloVe provides dense vector representations for words
- It is trained on the Wikipedia corpus and obtained based on how frequently words co-occur with one another
- The vector size is 50 (dimensions)

# 50 dimensional GloVe vector for 'King'

```
[ 0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 ,
-0.08813 , 0.47377 , -0.61798 , -0.31012 , -0.076666, 1.493 ,
-0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 ,
-0.55809 , 0.66421 , 0.1961 , -0.13495 , -0.11476 , -0.30344 ,
0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 ,
1.9927 , -0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 ,
0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 ,
-1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 ,
-0.64426 , -0.51042 ]
```

# Visualizing the GloVe vector for 'King'

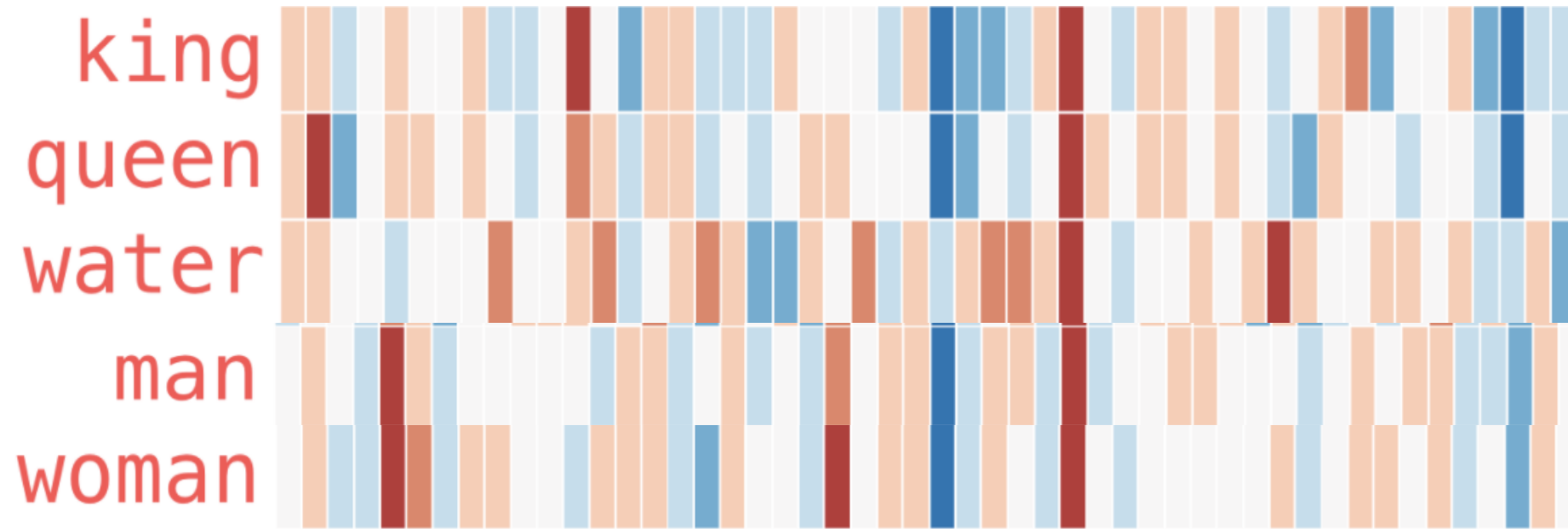
## Color code



Color coded vector representation of 'king'  
red if the number is close to 2, white if close to 0, blue if close to -2

[Jay Alammur, Illustrated Word2Vec](#)

# Comparison of the color-coded GloVe vectors



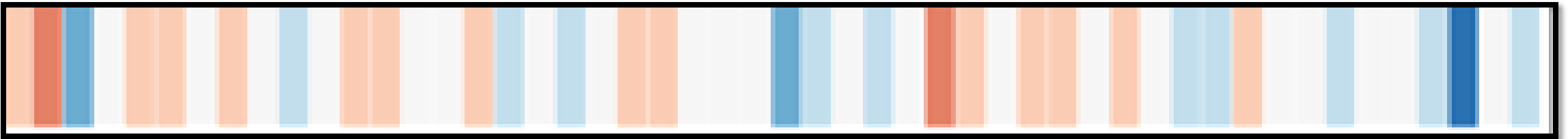
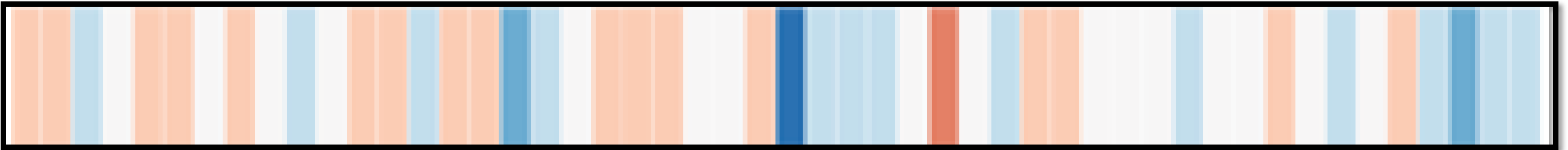
Jay Alammur, Illustrated Word2Vec

# Word vector analogies

- We can add and subtract word embeddings and arrive at interesting results
- The most famous example is the formula: “king” - “man” + “woman” = ~ “queen”

king – man + woman  $\approx$  queen

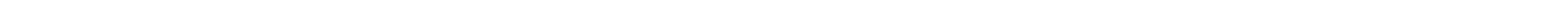
king – man + woman



queen

Jay Alammur, Illustrated Word2Vec

# Word vectors - Word2Vec





# Word2Vec pre-trained model

- Google's pre-trained Word2Vec model includes word vectors for a vocabulary of 3 million words and phrases
- Trained on roughly 100 billion words from a Google News dataset
- The vector size is 300 (dimensions)

# Gensim Python Package

- Serves as an interface to Word2Vec
- Makes it easy to load the Word2Vec model

```
import gensim
```

```
# Load Google's pre-trained Word2Vec model
```

```
model = gensim.models.Word2Vec.load_word2vec_format('./model/GoogleNews-vectors-negative300.bin')
```

# 300 dimensional Word2Vec vector for 'King'

[ 1.25976562e-01, 2.97851562e-02, 8.60595703e-03, 1.39648438e-01,  
-2.56347656e-02, -3.61328125e-02, 1.11816406e-01, -1.98242188e-01,  
5.12695312e-02, 3.63281250e-01, -2.42187500e-01, -3.02734375e-01,  
-1.77734375e-01, -2.49023438e-02, -1.67968750e-01, -1.69921875e-01,  
3.46679688e-02, 5.21850586e-03, 4.63867188e-02, 1.28906250e-01,  
1.36718750e-01, 1.12792969e-01, 5.95703125e-02, 1.36718750e-01,  
.....  
-2.73437500e-01, -3.07617188e-02, -3.37890625e-01, 1.53320312e-01,  
2.33398438e-01, -2.08007812e-01, 3.73046875e-01, 8.20312500e-02,  
2.51953125e-01, -7.61718750e-02, -4.66308594e-02, -2.23388672e-02,  
2.99072266e-02, -5.93261719e-02, -4.66918945e-03, -2.44140625e-01,  
-2.09960938e-01, -2.87109375e-01, -4.54101562e-02, -1.77734375e-01,  
-2.79296875e-01, -8.59375000e-02, 9.13085938e-02, 2.51953125e-01]

# Vector similarity

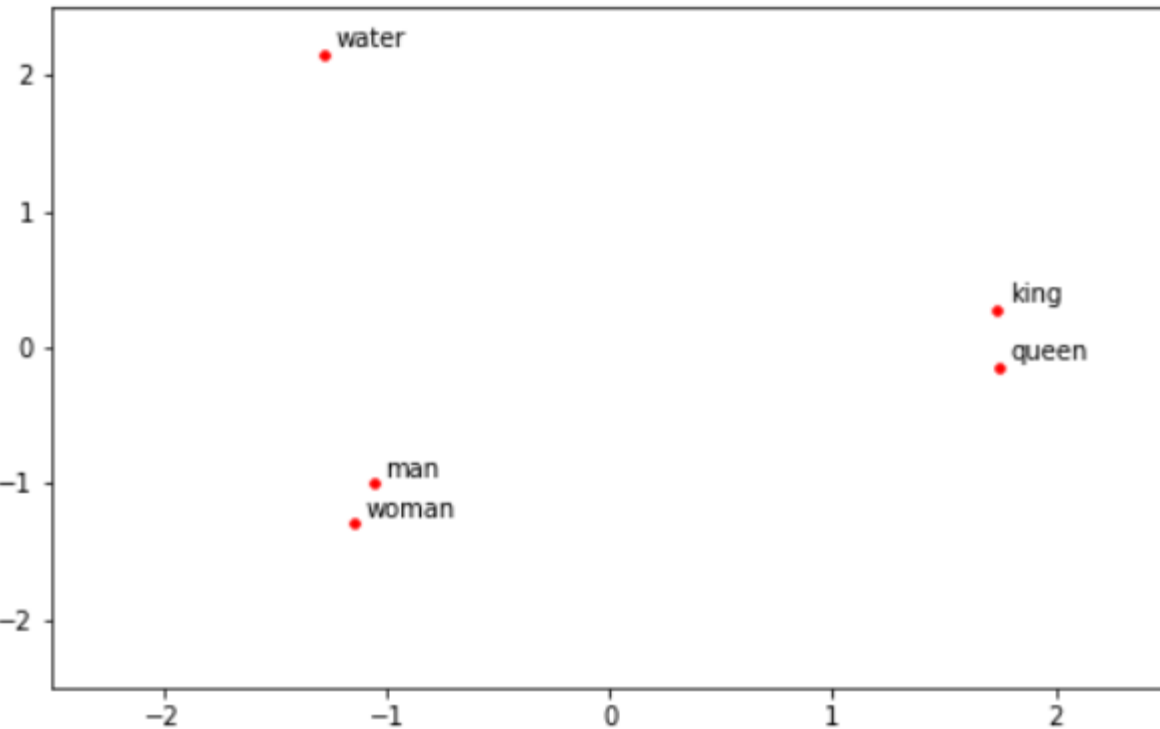
Euclidean distance of words for word2vec representation

|       | king     | queen    | water    | man      | woman    |
|-------|----------|----------|----------|----------|----------|
| king  | 0.000000 | 2.479692 | 3.821717 | 3.268789 | 3.673551 |
| queen | 2.479692 | 0.000000 | 3.949938 | 3.489749 | 3.337286 |
| water | 3.821717 | 3.949938 | 0.000000 | 3.343232 | 3.516967 |
| man   | 3.268789 | 3.489749 | 3.343232 | 0.000000 | 1.727951 |
| woman | 3.673551 | 3.337286 | 3.516967 | 1.727951 | 0.000000 |

Cosine similarity of words for word2vec representation

|       | king     | queen    | water    | man      | woman    |
|-------|----------|----------|----------|----------|----------|
| king  | 1.000000 | 0.651096 | 0.035606 | 0.229427 | 0.128480 |
| queen | 0.651096 | 1.000000 | 0.018264 | 0.166582 | 0.316181 |
| water | 0.035606 | 0.018264 | 1.000000 | 0.073456 | 0.101797 |
| man   | 0.229427 | 0.166582 | 0.073456 | 1.000000 | 0.766401 |
| woman | 0.128480 | 0.316181 | 0.101797 | 0.766401 | 1.000000 |

# Visualization of Word2Vec vectors in 2D



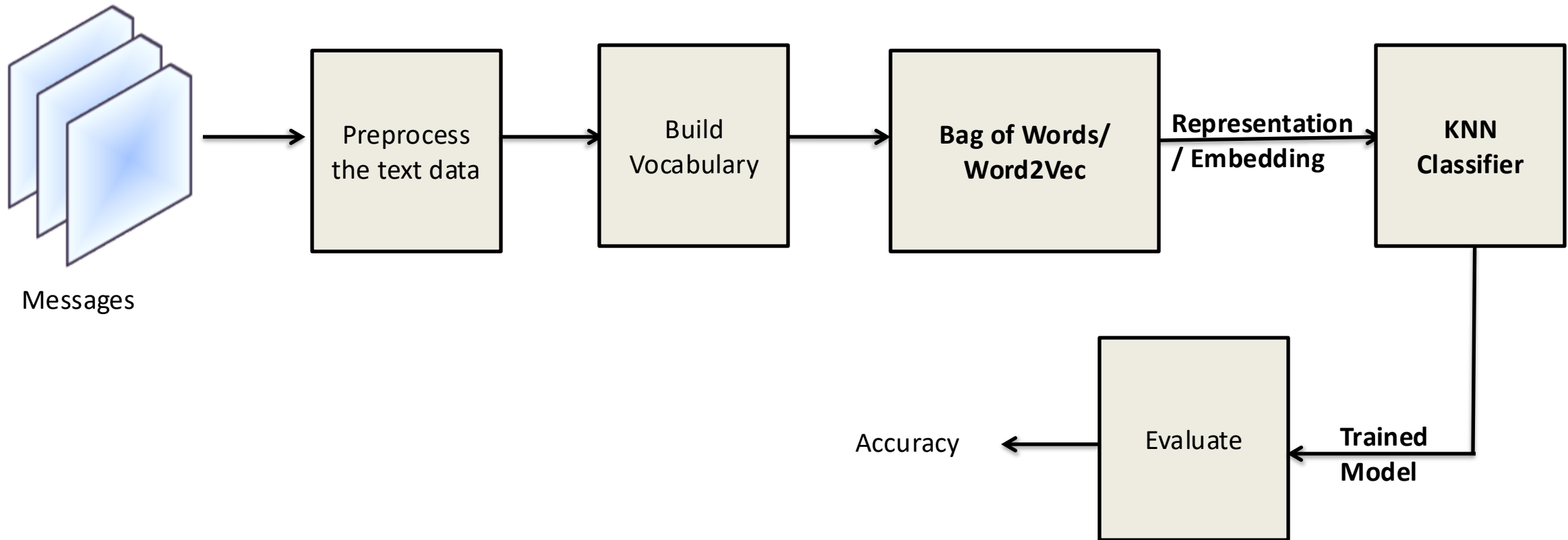
# Word\_Similarity\_Demo experiment

- Obtain GloVe/ Word2Vec word vectors
- Identify similar words using Cosine similarity
- Visualize similar words in 2D plot

# Mahabharata\_Demo\_experiment

- Train a Word2Vec model on the Mahabharata Epic's text corpus
- Identify similarities in Mahabharata characters through vector similarities

# Case Study: BoW vs W2V

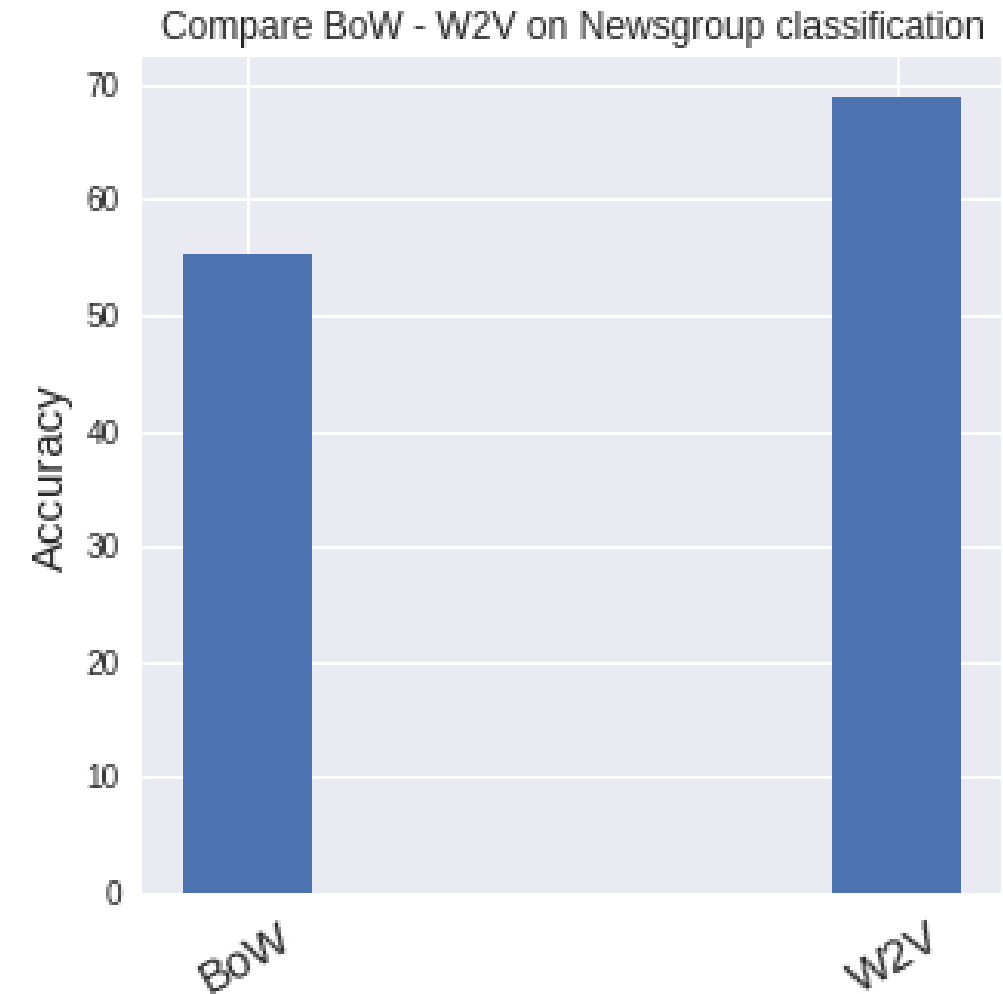


## PIPELINE

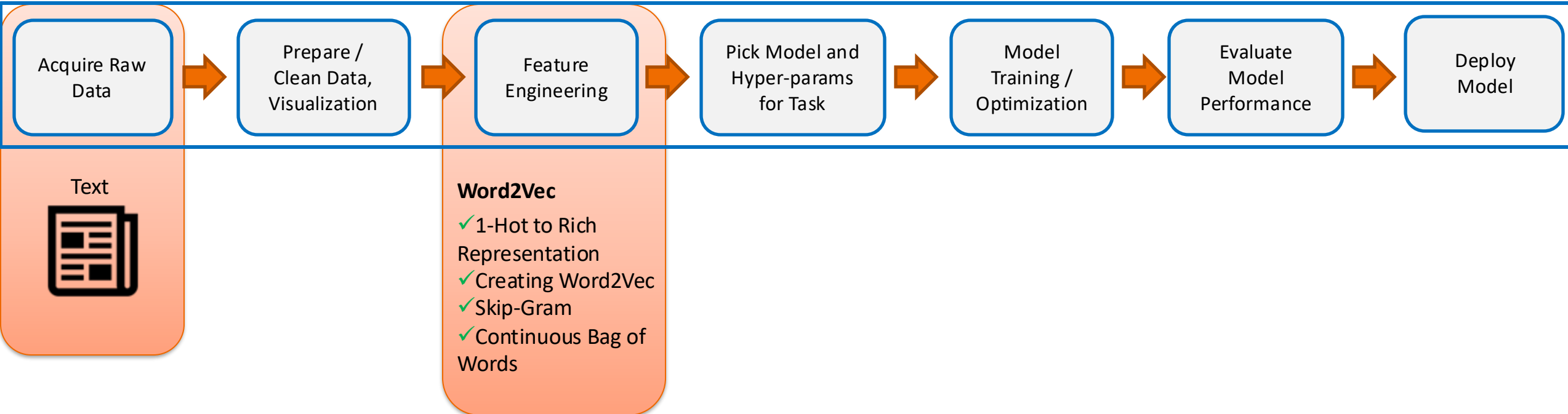


## BoW vs W2V

- Words with similar or different meanings are equally apart in BoW
- Semantics or Meaning of the word is preserved in W2V



# Summary



# Questions?